代表的な深層生成モデルとその分類

The Principles of Diffusion Models, Ch. 1 §1.2– §1.3 (pp. 23–27)

八谷研究室 輪講ゼミ

2026年10月15日（木）

この発表で扱う範囲

出典：C.-H. Lai, Y. Song, D. Kim, Y. Mitsufuji, S. Ermon, The Principles of Diffusion Models, arXiv:2510.21890, 第1章 1.2節・1.3節（p.23–27）

1.2–1.3節の主張

前回の壁「**正規化定数 $Z(\phi)$ が計算できない**」に対して，
代表的な深層生成モデル（EBM・AR・VAE・NF・GAN）は**それぞれ別の方法**で対処している。
それらは $p_\phi(\mathbf{x})$ を**明示的に書くか／暗黙的に定めるか**で分類でき，拡散モデルはその考え方を受け継ぐ。

前回（1.1節）とのつながり

前回：p_{data} に近い p_ϕ を，ダイバージェンス $\mathcal{D}(p_{\text{data}}, p_\phi)$ の最小化で学習したい　→　今回：**実際に p_ϕ をどう作り，どう学習するか**の具体例

目次

	節	主な内容	スライド
1	1.2節 導入 本文 p.23	3つのトレードオフ，$Z(\phi)$ への対処という視点	4–
2	エネルギーベースモデル (EBM) 本文 p.23	定義，スコアで Z が消える	5–
3	自己回帰モデル (AR) 本文 p.23–24	連鎖律，積の正規化，学習は並列・生成は逐次	8–
4	変分オートエンコーダ (VAE) 本文 p.24–25	ELBO の導出	14–
5	正規化フロー (NF) 本文 p.25	変数変換公式の証明，多次元の補正 $\lvert\det\rvert$	18–
6	敵対的生成ネットワーク (GAN) 本文 p.25–26	最適識別器，JS ダイバージェンスへの帰着，f-GAN	23–
7	1.3節 モデルの分類 本文 p.27	明示的／暗黙的モデル，表 1.1，拡散モデルとのつながり	31–

1.2 代表的な深層生成モデル

生成モデリングの中心課題

高次元データの複雑な構造を表現できる確率モデル p_ϕ を，データから学習すること

どのモデルも，次の3つの性質の間で**トレードオフ**をとっている

① 扱いやすさ

tractability
対数尤度 $\log p_\phi(\mathbf{x})$ とその勾配 $\nabla_\phi \log p_\phi(\mathbf{x})$ 等が**計算できるか**

② 表現力

expressiveness
複雑な分布を表せるか
（モデル構造への制約が少ないか）

③ 学習効率

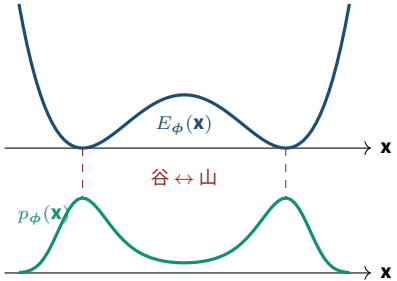
training efficiency
学習が安定して，現実的な計算量で終わるか

エネルギーベースモデル (EBM)

定義

$$p_\phi(\mathbf{x}) := \frac{1}{Z(\phi)} \exp\bigl(-E_\phi(\mathbf{x})\bigr), \qquad Z(\phi) := \int \exp\bigl(-E_\phi(\mathbf{x})\bigr)\, d\mathbf{x} \quad \text{(分配関数)}$$

- $E_\phi(\mathbf{x})$：**エネルギー関数**．ニューラルネットで作り，出力はスカラー1つ（前回 1.1.2 より）
- $E_\phi(\mathbf{x})$ は負にもなるが，$\exp(-E_\phi(\mathbf{x}))$ は必ず正（条件 (i)）
  $Z(\phi)$ で割って積分を1にする（条件 (ii)）
  前回 1.1.2 の Step 1・Step 2 と同じ（符号だけ逆）
- $\exp(-\cdot)$ は単調減少なので
  エネルギーが低い $\iff$ 確率が高い
- 学習：対数尤度の最大化（前回 Lemma 1.1.1）
  $\rightarrow$ しかし $Z(\phi)$ が**計算できない**



エネルギーの谷に確率が集まる

補足：スコアでは $Z(\phi)$ が消える

命題

EBM のスコア関数（$\mathbf{x}$ についての勾配）は $Z(\phi)$ によらない：　$\nabla_{\mathbf{x}} \log p_\phi(\mathbf{x}) = -\nabla_{\mathbf{x}} E_\phi(\mathbf{x})$

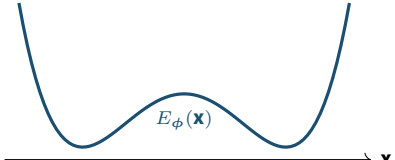
証明

$$\log p_\phi(\mathbf{x}) = -E_\phi(\mathbf{x}) - \log Z(\phi)$$

$Z(\phi) = \int \exp(-E_\phi(\mathbf{x}'))\, d\mathbf{x}'$ は $\mathbf{x}'$ について**積分し終えた数**なので，ϕ だけの関数で $\mathbf{x}$ を含まない

よって $\nabla_{\mathbf{x}} \log Z(\phi) = \mathbf{0}$ となり

$$\nabla_{\mathbf{x}} \log p_\phi(\mathbf{x}) = -\nabla_{\mathbf{x}} E_\phi(\mathbf{x}) \qquad \blacksquare$$



赤矢印：$-\dfrac{dE}{dx}$（1次元のスコア）

$\Rightarrow$ **Z を知らなくても確率の高い方向がわかる**

EBM を①②③で見る

観点	評価	理由
① 扱いやすさ	×	$Z(\phi)$ が計算できず $\log p_\phi(\mathbf{x})$ が求まらない
② 表現力	◎	E_ϕ はスカラーを1つ出力すればよく，ネットワークの形に制約がない
③ 学習効率	△	$Z(\phi)$ が計算できないことへの工夫が必要で，学習が重く難しい（本文）

本文の予告：拡散モデルとのつながり

拡散モデルは**対数密度の勾配（スコア）**を使ってデータを生成する．
スコアは $Z(\phi)$ によらない（スライド6）ので，**分配関数の計算そのものを回避できる**．

自己回帰モデル (AR)

確率の連鎖律（任意の同時分布で成り立つ）

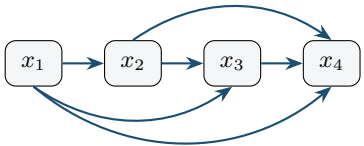
$$p_{\text{data}}(\mathbf{x}) = \prod_{i=1}^{D} p_{\text{data}}(x_i \mid x_{<i}), \qquad x_{<i} := (x_1, \ldots, x_{i-1}), \quad \mathbf{x} = (x_1, \ldots, x_D)$$

これは**モデルの仮定ではなく恒等式**．変数をどの順番に並べても成り立つ（証明 → 補足）

- 真の条件付き分布 $p_{\text{data}}(x_i \mid x_{<i})$ は分からない
- そこで**同じ分解の形**のモデルを作る：

$$p_\phi(\mathbf{x}) = \prod_{i=1}^{D} p_\phi(x_i \mid x_{<i})$$

- 各条件付き分布をニューラルネット（例：Transformer）で近似
- 出力の形：
  - 離散データ → softmax でカテゴリ分布
  - 連続データ → 扱いやすい密度のパラメータ（例：正規分布の平均・分散）



x_i は**それより前のすべて** $x_{<i}$ に依存して決まる

（$D = 4$ の例）

$$p_\phi(\mathbf{x}) = p_\phi(x_1)\, p_\phi(x_2|x_1) \times p_\phi(x_3|x_1, x_2)\, p_\phi(x_4|x_1, x_2, x_3)$$

補足：確率の連鎖律の証明 (1/2) — 準備と3変数の場合

命題（連鎖律）

任意の同時分布について $p(x_1, \ldots, x_D) = \prod_{i=1}^{D} p(x_i \mid x_{<i})$　（$i = 1$ の項は $p(x_1)$ と読む）

記号の意味

$p(x_1, \ldots, x_D)$：**同時分布**（全部が同時にその値をとる確率）
$p(a \mid b)$：b がわかったうえでの a の確率
$x_{<i} = (x_1, \ldots, x_{i-1})$：$i$ 番目より前の全部

条件付き確率の定義

$p(a \mid b) := \frac{p(a,b)}{p(b)}$ の両辺に $p(b)$ を掛けて

$$p(a, b) = p(b)\, p(a \mid b) \quad (\star)$$

b は**いくつかの変数のかたまり**でもよい

3変数の場合（$D = 3$）

Step 1：$(\star)$ で $a = x_3$，$b = (x_1, x_2)$ とおく

$$p(x_1, x_2, x_3) = p(x_1, x_2)\, p(x_3 \mid x_1, x_2)$$

Step 2：残った $p(x_1, x_2)$ に，$(\star)$ を $a = x_2$，$b = x_1$ で使う

$$p(x_1, x_2) = p(x_1)\, p(x_2 \mid x_1)$$

Step 3：Step 2 を Step 1 に代入

$$p(x_1, x_2, x_3) = p(x_1)\, p(x_2 \mid x_1)\, p(x_3 \mid x_1, x_2)$$

補足：確率の連鎖律の証明 (2/2) — 一般の D（数学的帰納法）

方針：① $D=1$ で成り立つ　② $D-1$ 個で成り立つなら D 個でも成り立つ　$\Rightarrow$ すべての D で成り立つ

証明

① $D=1$：左辺も右辺も $p(x_1)$ なので成り立つ．

② $D-1 \Rightarrow D$

Step 1：$(\star)$ で $a=x_D$（最後の1個），$b=x_{<D}$（それより前の全部）

$$p(x_1,\ldots,x_D)=p(x_{<D})\,p(x_D\mid x_{<D})$$

Step 2：$p(x_{<D})=p(x_1,\ldots,x_{D-1})$ は $D-1$ 個の同時分布．
$D-1$ 個では成り立つ（仮定）ので $p(x_{<D})=\prod_{i=1}^{D-1}p(x_i\mid x_{<i})$

Step 3：Step 2 を Step 1 に代入

$$\begin{aligned}p(x_1,\ldots,x_D)&=\prod_{i=1}^{D-1}p(x_i\mid x_{<i})\cdot p(x_D\mid x_{<D})\\&=\prod_{i=1}^{D}p(x_i\mid x_{<i})\qquad\blacksquare\end{aligned}$$

順番は任意

証明は変数の**並べ方**を使っていないので，順番を変えても成り立つ：

$$p(x_1,x_2,x_3)=p(x_3)\,p(x_1|x_3)\,p(x_2|x_1,x_3)$$

ただし**どの順番を選ぶか**で，ニューラルネットが学ぶ条件付き分布は変わる

補足：条件付き分布の積は自動的に正規化される

命題

各 $p_{\phi}(x_i \mid x_{<i})$ が x_i について正規化されていれば，積 $p_{\phi}(\mathbf{x}) = \prod_i p_{\phi}(x_i \mid x_{<i})$ も正規化される：$\int p_{\phi}(\mathbf{x})\, d\mathbf{x} = 1$

証明（内側の x_D から順に積分する）

$$\int p_{\phi}(\mathbf{x})\, d\mathbf{x} = \int \cdots \int \prod_{i=1}^{D-1} p_{\phi}(x_i|x_{<i}) \underbrace{\left[\int p_{\phi}(x_D|x_{<D})\, dx_D\right]}_{=1} dx_{<D}$$

$$= \int \cdots \int \prod_{i=1}^{D-1} p_{\phi}(x_i|x_{<i})\, dx_{<D}$$

$$= \cdots = \int p_{\phi}(x_1)\, dx_1 = 1 \qquad \blacksquare$$

x_D を含む因子は最後の1つだけなので，その積分を先に実行できる．同じことを $x_{D-1}, x_{D-2}, \ldots$ と繰り返す．

$dx_{<D} := dx_1 \cdots dx_{D-1}$　離散の場合は $\int$ を $\sum$ に置き換えれば同じ

なぜ Z の問題が起きないのか

各条件付き分布を正規化しているため，全体の Z は不要

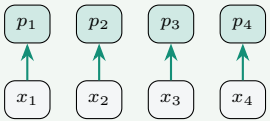
学習は並列，生成は逐次

対数をとると積が和になる：

$$\log p_{\phi}(\mathbf{x}) = \sum_{i=1}^{D} \log p_{\phi}(x_i \mid x_{<i}) \quad \Rightarrow \quad \text{そのまま最尤推定で学習できる}$$

学習：並列に計算できる

学習データ **x** は**すべて手元にある**
⇒ どの i でも条件 $x_{<i}$ が分かっている
⇒ D 個の $\log p_{\phi}(x_i|x_{<i})$ を**まとめて**計算できる



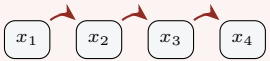
$p_1, \ldots, p_4$ を**まとめて同時に**計算できる（$p_i := p_{\phi}(x_i|x_{<i})$）

⇒ 長い系列では**生成が遅い**

生成：1つずつ順番に

まだ **x** がないので，前から順にサンプリングするしかない：

$$x_1 \sim p_{\phi}(x_1)$$
$$\rightarrow x_2 \sim p_{\phi}(x_2|x_1) \rightarrow \cdots$$
$$\rightarrow x_D \sim p_{\phi}(x_D|x_{<D})$$



ネットワークをD **回**順に呼ぶ必要がある

AR を①②③で見る

観点	評価	理由
① 扱いやすさ	◎	$\log p_\phi(\mathbf{x})$ が**厳密に**計算でき，勾配も自動微分でそのまま求まる
② 表現力	○	連鎖律は恒等式なので原理的にはどんな分布も表せる．ただし x_i が $x_{<i}$ だけを見るよう**順番の制約**がかかる
③ 学習効率	◎	普通の最尤推定で安定．学習時は並列に計算できる

①〜③とは別の弱点：生成速度

生成は D ステップの逐次計算になるので遅い（学習効率ではなく**生成**の問題）

本文より

AR モデルは尤度ベースの生成モデルの基礎であり，**現代の系列モデル・言語モデル**の土台になっている

（例：文章を「次の単語の予測」の繰り返しで生成する大規模言語モデル）

変分オートエンコーダ (VAE)

データ $\mathbf{x}$ の裏に，その**隠れた構造**を表す潜在変数 $\mathbf{z}$ があると考える

生成の仕組み（デコーダ側）

$$\mathbf{z} \sim p_{\text{prior}}(\mathbf{z}) \quad \rightarrow \quad \mathbf{x} \sim p_\phi(\mathbf{x} \mid \mathbf{z})$$

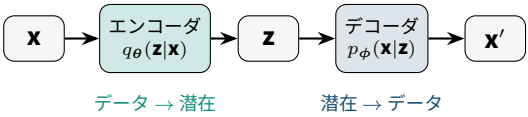
p_{prior}：単純な分布（多くは標準正規分布）

$p_\phi(\mathbf{x}|\mathbf{z})$：**デコーダ**（ニューラルネット）

困ること：尤度が計算できない

$$p_\phi(\mathbf{x}) = \int p_\phi(\mathbf{x} \mid \mathbf{z})\, p_{\text{prior}}(\mathbf{z})\, d\mathbf{z}$$

$\mathbf{z}$ についての積分の中にニューラルネットがあり，閉じた式にならない



2つのネットワークを同時に学習

エンコーダ $q_\theta(\mathbf{z}|\mathbf{x})$：$\mathbf{x}$ を生んだ $\mathbf{z}$ の分布を**推測**

（本当の事後分布 $p_\phi(\mathbf{z}|\mathbf{x})$ は分からないので近似する）

デコーダ $p_\phi(\mathbf{x}|\mathbf{z})$：$\mathbf{z}$ から $\mathbf{x}$ を**再構成**

VAE の学習目的：ELBO

尤度 $\log p_\phi(\mathbf{x})$ の代わりに，計算できる**下界**（Evidence Lower Bound）を最大化する

ELBO

$$\mathcal{L}_{\mathrm{ELBO}}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}) = \underbrace{\mathbb{E}_{q_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})}\left[\log p_{\boldsymbol{\phi}}(\mathbf{x} \mid \mathbf{z})\right]}_{\text{(a) 再構成}} - \underbrace{D_{\mathrm{KL}}\left(q_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x}) \,\|\, p_{\mathrm{prior}}(\mathbf{z})\right)}_{\text{(b) 正則化}} \leq \log p_{\boldsymbol{\phi}}(\mathbf{x})$$

(a) 再構成項

エンコーダで $\mathbf{z}$ を選び，デコーダで $\mathbf{x}$ に戻したとき，
元の $\mathbf{x}$ がよく再現されるほど大きい

(b) 正則化項

エンコーダの出す分布が**事前分布 p_{prior} から離れすぎない**ようにする
$\Rightarrow$ 生成時に p_{prior} から $\mathbf{z}$ を引けば，意味のある $\mathbf{x}$ が出る

なぜこれで良いのか

なぜ $\log p_\phi(\mathbf{x})$ 以下になるのか（下界であること） $\rightarrow$ 後の補足（Jensen の不等式）で示す

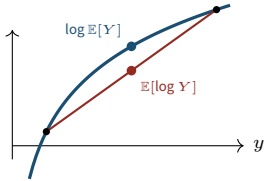
補足：ELBO の導出 — Jensen の不等式

Jensen の不等式

正の確率変数 Y について

$$\log \mathbb{E}[Y] \;\geq\; \mathbb{E}[\log Y]$$

理由：log は**上に凸**
（弦がグラフの下にある）



導出

$$\begin{aligned}
\log p_{\phi}(\mathbf{x}) &= \log \int p_{\phi}(\mathbf{x}|\mathbf{z})\, p_{\text{prior}}(\mathbf{z})\, d\mathbf{z} \\
&\overset{\text{(i)}}{=} \log \int q_{\theta}(\mathbf{z}|\mathbf{x})\, \frac{p_{\phi}(\mathbf{x}|\mathbf{z})\, p_{\text{prior}}(\mathbf{z})}{q_{\theta}(\mathbf{z}|\mathbf{x})}\, d\mathbf{z} \\
&= \log \mathbb{E}_{q_{\theta}(\mathbf{z}|\mathbf{x})}\left[\frac{p_{\phi}(\mathbf{x}|\mathbf{z})\, p_{\text{prior}}(\mathbf{z})}{q_{\theta}(\mathbf{z}|\mathbf{x})}\right] \\
&\overset{\text{(ii)}}{\geq} \mathbb{E}_{q_{\theta}(\mathbf{z}|\mathbf{x})}\left[\log \frac{p_{\phi}(\mathbf{x}|\mathbf{z})\, p_{\text{prior}}(\mathbf{z})}{q_{\theta}(\mathbf{z}|\mathbf{x})}\right] \\
&= \mathbb{E}_{q_{\theta}}\left[\log p_{\phi}(\mathbf{x}|\mathbf{z})\right] - \mathbb{E}_{q_{\theta}}\left[\log \frac{q_{\theta}(\mathbf{z}|\mathbf{x})}{p_{\text{prior}}(\mathbf{z})}\right] \\
&= \mathbb{E}_{q_{\theta}}\left[\log p_{\phi}(\mathbf{x}|\mathbf{z})\right] - D_{\text{KL}}\left(q_{\theta}(\mathbf{z}|\mathbf{x}) \,\|\, p_{\text{prior}}(\mathbf{z})\right) = \mathcal{L}_{\text{ELBO}} \quad \blacksquare
\end{aligned}$$

(i) $q_{\theta}/q_{\theta} = 1$ を掛けた　(ii) Jensen の不等式

ポイント：計算できない積分を「q_{θ} についての期待値」に書き換え，log を期待値の内側に入れた

VAE を①②③で見る

観点	評価	理由
① 扱いやすさ	△	$\log p_\phi(\mathbf{x})$ は計算できない．計算できるのは**下界**（ELBO）とその勾配
② 表現力	○	エンコーダ・デコーダとも形は自由．ただしサンプルが**ぼやけやすい**（本文）
③ 学習効率	◎	ELBO を確率的勾配法でそのまま最大化でき，安定して学習できる

本文が挙げる弱点

・生成サンプルの鮮明さに限界がある　・学習の病理：デコーダが**潜在変数を無視**してしまうことがある

本文より：拡散モデルとのつながり

VAE はニューラルネットと潜在変数モデルを理論的に結びつけ，**拡散モデルを含む後の発展の土台**になった（ELBO による学習は 2章の DDPM で再登場する）

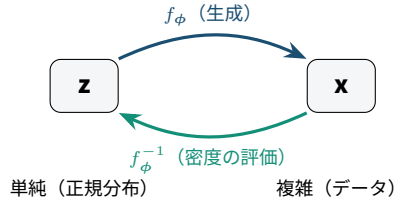
正規化フロー (NF)

単純な分布の **z** を，**可逆な変換** f_ϕ で複雑なデータ **x** に写す

変数変換公式で密度が厳密に書ける

$$\log p_\phi(\mathbf{x}) = \log p(\mathbf{z}) + \log \left| \det \frac{\partial f_\phi^{-1}(\mathbf{x})}{\partial \mathbf{x}} \right|$$

$\mathbf{z} = f_\phi^{-1}(\mathbf{x})$，$p(\mathbf{z})$：単純な分布（例：標準正規分布）
det：ヤコビ行列（偏微分を並べた $D \times D$ 行列）の行列式



- 正規化は公式の中で**自動的に**保たれる
  $\Rightarrow Z$ が不要，**尤度が厳密に計算できる**
- そのまま最尤推定で学習できる

可逆変換の作り方：2通り

NF：可逆な変換を**重ねる**
$f_\phi = f_L \circ \cdots \circ f_2 \circ f_1$

Neural ODE：変換を**常微分方程式**で表す
$\frac{d\mathbf{x}(t)}{dt} = \mathbf{v}_\phi(\mathbf{x}(t), t)$ を解く（例：$t = 0 \to 1$）

$\mathbf{v}_\phi$：各位置での速度（どっちにどれだけ動くか）を決めるニューラルネット．**z** から少しずつ動かして **x** に着く
（逆にたどれば戻れる＝可逆）

補足：累積分布関数とは

定義

$$F_X(x) := P(X \le x) \qquad (X \text{ が } x \textbf{ 以下}\text{になる確率})$$

左から確率を**積み上げて（累積して）**いったもの $\Rightarrow$ x が大きいほど増え，最後は 1 になる

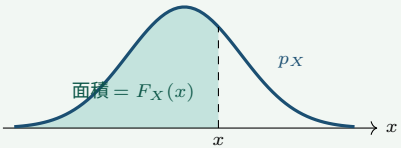
例：さいころの目 X

x	$F_X(x) = P(X \le x)$
1	$1/6$
2	$2/6$
3	$3/6$
$\vdots$	$\vdots$
6	1

連続の場合：密度との関係

$F_X(x)$ ＝ 密度のグラフの，x より**左側の面積**

$$F_X(x) = \int_{-\infty}^{x} p_X(t)\,dt \quad \Longleftrightarrow \quad p_X(x) = \frac{dF_X(x)}{dx}$$



$\Rightarrow$ 次のページでは，密度を直接変換する代わりに，**まず F を変換してから微分**して密度を求める

補足：変数変換公式の証明（1次元）

$Z \sim p_Z$，$X = f(Z)$，f は**狭義単調増加**で微分可能とする

狭義単調増加：逆関数 f^{-1} を使うため（1次元では本文の「可逆」と同じ意味）　微分可能：密度 = 累積分布関数の微分 なので

証明（累積分布関数を経由する）

$$F_X(x) := P(X \le x) = P\big(f(Z) \le x\big)$$
$$= P\big(Z \le f^{-1}(x)\big) = F_Z\big(f^{-1}(x)\big)$$

両辺を x で微分（合成関数の微分）して

$$p_X(x) = p_Z\big(f^{-1}(x)\big)\,\frac{df^{-1}(x)}{dx}$$

f が減少関数なら不等号が逆向きになり，符号が変わる

$\Rightarrow$ まとめて $\left|\dfrac{df^{-1}(x)}{dx}\right|$ ■

対数をとると本文の式（$D = 1$ の場合）になる

NF では z は単純な分布（例：正規分布），f_ϕ は自作のニューラルネット

$\Rightarrow$ $p_\phi(\mathbf{x})$ **が正確に求まり**，そのまま最尤推定できる

この公式が言っていること

z の分布と変換 f がわかれば，x の密度が**正確に**計算できる

$$p_X(x) = \underbrace{p_Z\big(f^{-1}(x)\big)}_{\text{① 元の場所の密度}} \times \underbrace{\left|\frac{df^{-1}(x)}{dx}\right|}_{\text{② 伸び縮みの補正}}$$

① x は変換前どの z だったかを逆算し，その z での密度を見る

② 変換で空間が何倍に伸びたかを見て調整する（**伸びた所は密度を下げる**）

多次元でも成り立つ（補正が $|\det|$ になる：次ページ）．

証明は重積分の変数変換を使うので，ここでは1次元で示す

補足：多次元の場合

多次元：補正＝体積の拡大率

例：$A = \begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$（横2倍・縦3倍）

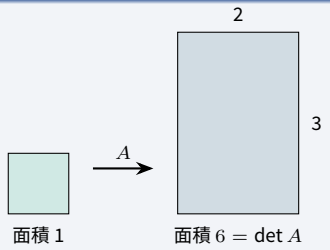
1×1 の正方形（面積1）$\rightarrow$ 2×3 の長方形（面積6）

$\det A = 2 \times 3 = 6$

$\Rightarrow |\det A|$ ＝ **面積（体積）の倍率**

曲がった変換 f でも，1点の近くでは**ヤコビ行列**（偏微分を並べた行列）で近似できる

$\Rightarrow$ 1次元の $|df^{-1}/dx|$（長さの倍率）が
$\left|\det(\partial f^{-1}/\partial \mathbf{x})\right|$（体積の倍率）になる



NF を①②③で見る

観点	評価	理由
① 扱いやすさ	◎	変数変換公式で $\log p_\phi(\mathbf{x})$ が**厳密に**計算できる
② 表現力	△	可逆性（$\mathbf{z}$ と $\mathbf{x}$ が同じ次元）と，行列式が簡単に（速く）計算できることが必要 $\Rightarrow$ 使える層の形が**強く制限**される（本文）
③ 学習効率	○／△	NF は最尤推定でそのまま学習できる． Neural ODE は学習のたびに**ODE を解く計算コスト**が重い（本文）

本文より

どちらの方式も**高次元データへのスケール**に課題がある

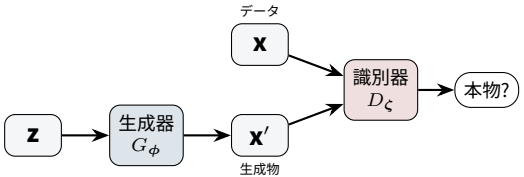
拡散モデルとのつながり

後の章で，拡散モデルがこれらのフロー型モデルとどう関係し，どう発展させたかを扱う（1.3節：NF は 5.1節で詳しく扱う）

敵対的生成ネットワーク (GAN)

2つのニューラルネットを**競わせて**学習する

- **生成器** G_ϕ：ノイズ $\mathbf{z} \sim p_{\text{prior}}$ から
  本物らしいサンプル $G_\phi(\mathbf{z})$ を作る
- **識別器** D_ζ：入力が本物か生成物かを見分ける
  （$D_\zeta(\mathbf{x}) \in (0,1)$ は「本物である確率」の推定）



目的関数（min-max）

$$\min_{G_\phi} \max_{D_\zeta} \underbrace{\mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\left[\log D_\zeta(\mathbf{x})\right]}_{\text{本物を本物と判定}} + \underbrace{\mathbb{E}_{\mathbf{z}\sim p_{\text{prior}}}\left[\log\left(1 - D_\zeta(G_\phi(\mathbf{z}))\right)\right]}_{\text{生成物を偽物と判定}}$$

識別器はこの値を**大きく**（見分けたい），生成器は**小さく**（だましたい）する
⇒ **密度 $p_\phi(\mathbf{x})$ の式を持たず**サンプルを作るだけなので，尤度推定・正規化定数を完全に回避

補足：最適な識別器の導出 (1/2)

生成器 G_ϕ を固定し，p_{G_ϕ} を生成物 $G_\phi(\mathbf{z})$ の分布とする．目的関数を $V(G,D)$ と書く

$$V(G,D) = \underbrace{\mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\left[\log D(\mathbf{x})\right]}_{\mathbf{x}\text{ についての平均}} + \underbrace{\mathbb{E}_{\mathbf{z}\sim p_{\text{prior}}}\left[\log\left(1 - D(G_\phi(\mathbf{z}))\right)\right]}_{\mathbf{z}\text{ についての平均}}$$

Step 1：2つの項を同じ $\mathbf{x}$ の積分にそろえる

(i) 「$\mathbf{z} \sim p_{\text{prior}}$ を引いて $G_\phi(\mathbf{z})$ を作る」＝「$\mathbf{x} \sim p_{G_\phi}$ を引く」（p_{G_ϕ} の定義）
⇒ 第2項は p_{G_ϕ} についての平均と書ける

$$\mathbb{E}_{\mathbf{z}\sim p_{\text{prior}}}\left[\log\left(1 - D(G_\phi(\mathbf{z}))\right)\right] = \mathbb{E}_{\mathbf{x}\sim p_{G_\phi}}\left[\log\left(1 - D(\mathbf{x})\right)\right]$$

(ii) 期待値は「密度 × 値」の積分：$\mathbb{E}_{\mathbf{x}\sim p}[g(\mathbf{x})] = \int p(\mathbf{x})\, g(\mathbf{x})\, d\mathbf{x}$

(iii) どちらも $\mathbf{x}$ の積分になったので，1つにまとめる

$$V(G,D) = \int \Big[p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) + p_{G_\phi}(\mathbf{x}) \log\left(1 - D(\mathbf{x})\right) \Big] d\mathbf{x}$$

補足：最適な識別器の導出 (2/2)

前ページ：$V(G,D) = \int \left[p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) + p_{G_\phi}(\mathbf{x}) \log(1 - D(\mathbf{x})) \right] d\mathbf{x}$

Step 2：各点 $\mathbf{x}$ ごとに最大化する

$a = p_{\text{data}}(\mathbf{x})$, $b = p_{G_\phi}(\mathbf{x})$，$y = D(\mathbf{x})$ とおき，
$h(y) = a \log y + b \log(1 - y)$　を最大化
（$\log y$, $\log(1-y)$ が定義されるため $0 < y < 1$）

$$h'(y) = \frac{a}{y} - \frac{b}{1-y} = 0 \iff y = \frac{a}{a+b}$$

$h''(y) = -\frac{a}{y^2} - \frac{b}{(1-y)^2} < 0$ なので最大．よって

$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_{G_\phi}(\mathbf{x})} \qquad \blacksquare$$

意味

- 本物が多い場所ほど $D^* \approx 1$，生成物が多い場所ほど $D^* \approx 0$
- $p_{G_\phi} = p_{\text{data}}$ なら
  どこでも $D^* = \frac{1}{2}$
  （**見分けがつかない**）

※ 積分の中身を各点で最大にすれば積分全体も最大になる

補足：最適識別器を代入すると JS ダイバージェンスになる

目標：前ページの D^* を代入して，本文の最小化問題の形になることを確かめる

$$\min_G \max_D V(G,D) \underset{\text{前ページ}}{=} \min_G V(G,D^*) \underset{\text{このページ}}{=} \min_G \; 2\,D_{\mathrm{JS}}(p_{\text{data}} \,\|\, p_{G_\phi}) - \log 4$$

計算（$m := \frac{1}{2}(p_{\text{data}} + p_{G_\phi})$：2つの分布の平均）

Step 1 代入 $\quad V(G,D^*) = \mathbb{E}_{p_{\text{data}}}\left[\log D^*\right] + \mathbb{E}_{p_{G_\phi}}\left[\log(1-D^*)\right] = \mathbb{E}_{p_{\text{data}}}\left[\log \frac{p_{\text{data}}}{2m}\right] + \mathbb{E}_{p_{G_\phi}}\left[\log \frac{p_{G_\phi}}{2m}\right]$

Step 2 $\log 2$ を外へ $\quad = \mathbb{E}_{p_{\text{data}}}\left[\log \frac{p_{\text{data}}}{m}\right] + \mathbb{E}_{p_{G_\phi}}\left[\log \frac{p_{G_\phi}}{m}\right] - 2\log 2$

Step 3 KL の形 $\quad = D_{\mathrm{KL}}(p_{\text{data}} \,\|\, m) + D_{\mathrm{KL}}(p_{G_\phi} \,\|\, m) - \log 4$

Step 4 JS の定義 $\quad = 2\,D_{\mathrm{JS}}(p_{\text{data}} \,\|\, p_{G_\phi}) - \log 4$ ■

各ステップで使ったこと

Step 1：$D^* = \frac{p_{\text{data}}}{p_{\text{data}}+p_{G_\phi}} = \frac{p_{\text{data}}}{2m}$，$1 - D^* = \frac{p_{G_\phi}}{2m}$

Step 2：$\log \frac{u}{2} = \log u - \log 2$（定数は平均の外に出せる）

Step 3：KL の定義 $D_{\mathrm{KL}}(p\|q) = \mathbb{E}_p\left[\log \frac{p}{q}\right]$

Step 4：$D_{\mathrm{JS}}(p\|q) := \frac{1}{2}D_{\mathrm{KL}}\left(p \,\|\, \frac{p+q}{2}\right) + \frac{1}{2}D_{\mathrm{KL}}\left(q \,\|\, \frac{p+q}{2}\right)$（本文の定義）

$p = p_{\text{data}}$，$q = p_{G_\phi}$ を入れると $\frac{p+q}{2} = m$．両辺を2倍して

$2\,D_{\mathrm{JS}}(p_{\text{data}}\|p_{G_\phi}) = D_{\mathrm{KL}}(p_{\text{data}}\|m) + D_{\mathrm{KL}}(p_{G_\phi}\|m)$

結論

識別器が最適なとき
生成器が V を小さくする
$=$ **本物とのずれ（JS）を小さくする**

学習の式 V に JS は出てこない
⇒「暗に」最小化

ゴール：生成分布を p_{data} に近づけること

f-GAN：GAN もダイバージェンス最小化の仲間

本文 p.26 の主張

f-GAN は，敵対的学習で JS 以外の「ずれ」（**f-ダイバージェンス**）も最小化できることを示した
$\Rightarrow$ GAN も他の生成モデルと同じ**「ダイバージェンス最小化」の枠組み**に入る

学習のしかた	小さくしている「ずれ」	どこで見たか
最尤推定（AR・NF など）	順方向 KL ダイバージェンス	前回 Lemma 1.1.1
GAN	JS ダイバージェンス	前ページ
f-GAN	f-ダイバージェンス（KL・JS などの仲間）	本文 p.26

f-ダイバージェンスとは（前回 1.1節）

2つの分布のずれの測り方の**仲間**．式の中の関数 f を取り替えると，KL や JS になる

GAN を①②③で見る

観点	評価	理由
① 扱いやすさ	×	密度を定義しないので，$\log p_\phi(\mathbf{x})$ はそもそも存在しない
② 表現力	◎	生成器はノイズを入れてデータを出すだけで，形の制約がない
③ 学習効率	×	min-max の学習が**非常に不安定**．設計や工夫が必要（本文）

本文より

- **高品質なデータ**を生成できる
- その後，他の生成モデル，特に**拡散モデルを強化する補助要素**として再び使われている

まとめ：5つのモデルを「$Z(\phi)$ への対処法」で見る

前回（1.1.2）：$p_\phi(\mathbf{x}) = \exp(-E_\phi(\mathbf{x}))/Z(\phi)$ の分配関数 $Z(\phi) = \int \exp(-E_\phi(\mathbf{x}))\, d\mathbf{x}$ が**計算できない**

モデル	$Z(\phi)$ への対処	代償
EBM	Z を**そのまま残す**（学習時に工夫して回避）	尤度の計算・学習が難しい
AR	正規化済みの**1次元条件付き分布の積**にする	生成が逐次的で遅い
VAE	潜在変数を導入し，尤度の**下界**（ELBO）を最大化	尤度は近似のみ
NF	**可逆変換**と変数変換公式で正規化を自動的に保つ	構造の制約・高次元で重い
GAN	**密度を定義しない**（サンプルだけ作る）	尤度なし・学習が不安定

1.3節へ

どのモデルも「$p_\phi(\mathbf{x})$ を式として書くか，書かずにサンプリング手続きだけで定めるか」で性格が決まる
→ **明示的モデル／暗黙的モデル**という分類

まとめ：5つのモデルを①②③で並べる

モデル	① 扱いやすさ	② 表現力	③ 学習効率	主な弱点
EBM	×	◎	△	$Z(\phi)$ が計算できない
AR	◎	○	◎	生成が逐次的で遅い
VAE	△	○	◎	尤度は下界のみ・ぼやけやすい
NF	◎	△	○／△	可逆性による構造の制約
GAN	×	◎	×	尤度なし・学習が不安定

読み取れること

3つすべてが◎のモデルはない ⇒ どれかを良くすると別のどれかが悪くなる（**トレードオフ**）

例：① を取る（AR・NF）と構造や生成速度に制約，② を取る（EBM・GAN）と尤度を失う

※ 評価は本文の記述をもとにした発表者による整理

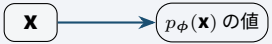
1.3 モデルの分類：明示的モデルと暗黙的モデル

学習の目的関数とは別に，$p_\phi(\mathbf{x})$ **をどう定めるか**で分類する

明示的モデル (explicit)

密度（確率）$p_\phi(\mathbf{x})$ を**式として**直接与える
（厳密に，または計算できる下界を通して）

例：AR，NF，VAE，拡散モデル (DM)



暗黙的モデル (implicit)

サンプリングの手続きだけで分布を定める：
$\mathbf{x} = G_\phi(\mathbf{z})$，$\mathbf{z} \sim p_{\text{prior}}$
$p_\phi(\mathbf{x})$ は式で書けず，定義できないこともある

例：GAN



	明示的		暗黙的
	厳密な尤度	近似的な尤度	
尤度	計算できる	下界・近似	直接はモデル化しない／計算できない
目的関数	最尤推定	ELBO	敵対的学習
例	NF，AR	VAE，DM	GAN

本文 表 1.1 をもとに作成．EBM は表にないが，密度を式で与えるので明示的モデル（尤度の計算は困難）

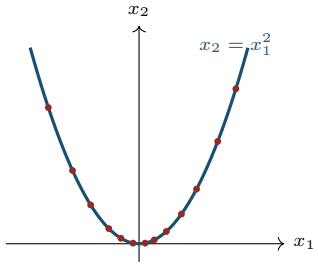
補足：暗黙的モデルでは密度が「存在しない」こともある

例：1次元ノイズ $\to$ 2次元データ

ノイズ：乱数1つ $z \sim \mathcal{N}(0,1)$
生成器：$G(z) = (z,\ z^2)$（横 $= z$，縦 $= z^2$ の点を作る）

作られる点は**全部，放物線（線）の上**に来る
線には幅がない $\Rightarrow$ 面積がない
密度 $=$ 点の割合 $\div$ 面積 なので，
　線の上では**0 で割る**ことになる
$\Rightarrow$ 密度 $p(\mathbf{x})$ を数で表せない

一般に，**ノイズの次元がデータより低い**と
生成物は線や薄い面の上にしか乗らない
（次元 $=$ 中に入っている数の個数．GAN ではふつう，ノイズはデータより次元が低い）



比較：NF の場合

NF は可逆変換なので，**ノイズとデータの次元は同じ**
$\Rightarrow$ 密度が**必ず存在し**，変数変換公式で求まる

1.3 拡散モデルとのつながり

これらの古典的なモデルは，複雑な分布を表すための**相補的な戦略**であり，拡散モデルを理解する指針にもなる

受け継ぐ元	拡散モデルが受け継ぐ考え方	今回の発表での伏線	本書
VAE	変分的な学習目的（ELBO）	ELBO の導出	2.1節
EBM	スコアマッチング（対数密度の勾配を学ぶ）	スコアでは Z が消える	3.1節
NF	連続時間の変換	Neural ODE	5.1節

次回以降

この3つ（VAE・EBM・NF）を出発点に，現代の拡散モデルの基本原理を組み立てていく

全体のまとめ

1.2節：代表的な深層生成モデル

- EBM・AR・VAE・NF・GAN は，前回の壁「**正規化定数** $Z(\phi)$」にそれぞれ別の方法で対処している
- ① 扱いやすさ・② 表現力・③ 学習効率の間に**トレードオフ**がある
- 学習法もダイバージェンス最小化として統一的に見られる（最尤推定 $\leftrightarrow$ KL，GAN $\leftrightarrow$ JS）

1.3節：モデルの分類

- $p_\phi(\mathbf{x})$ を式で与える**明示的モデル**と，サンプリング手続きだけで定める**暗黙的モデル**
- 拡散モデルは VAE（ELBO）・EBM（スコア）・NF（連続時間変換）の考え方を受け継ぐ